

113. Path Sum II

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 31, 2021

Problem Summary

Given the root of a binary tree and targetSum, return all root-to-leaf paths where node values sum to targetSum.

Algo

1. If root == null return.
2. current.add(root.val).
3. If leaf and target == root.val → result.add(new ArrayList<>(current)).
4. Recurse left and right with target - root.val.
5. current.remove(current.size() - 1) — backtrack.

Time Complexity & Space Complexity

$O(n^2)$ — $O(n)$ nodes × $O(n)$ to copy path in worst case (skewed tree).

$O(h)$ — recursion stack + current path list, both $O(h)$.

Approach

DFS with backtracking — maintain a current path list. Add node on the way down, remove it on the way back up. When a valid leaf is found, add a **copy** of current path to result.

Key Insights

- new ArrayList<>(current) — must copy, not add reference. Direct reference means result changes when current is modified later.
- Backtrack step always runs — whether path was valid or not, remove the current node before returning to parent.
- Add before recurse, remove after recurse — the two backtracking lines always appear together.
- Same subtraction trick as LC 112 — target reduces going down.

Observations

- current.remove(current.size() - 1) removes by **index** — remove(int index). Using remove(root.val) would call remove(Object) — wrong behavior for integer lists.
- Backtrack step is outside the if block — always executes regardless of whether path was valid.
- This backtracking pattern is foundational — reused in all combination/permutation/subset problems.

Problem List

113. Path Sum II

Solved

Medium

Topics

Companies

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2],[5,8,4,5]]
Explanation: There are two paths whose sum equals targetSum:
5 + 4 + 11 + 2 = 22
5 + 8 + 4 + 5 = 22

Example 2:

Accepted

115 / 115 testcases passed

Sahil Khan submitted at Jul 31, 2026 22:25

Runtime

1 ms | Beats 99.97%

Memory

45.49 MB | Beats 57.64%

1ms 2ms 3ms 4ms

Code

Java

```
14 * }
15 */
16 class Solution {
17     public List<List<Integer>> pathSum(TreeNode root, int targetSum) {
18         List<List<Integer>> result = new ArrayList<>();
19         List<Integer> current = new ArrayList<>();
20         dfs(root, targetSum, current, result);
21         return result;
22     }
23
24     void dfs(TreeNode root, int target, List<Integer> current, List<List<Integer>> result) {
25         if(root == null) return;
26
27         current.add(root.val);
28
29         if(root.left == null && root.right == null && target == root.val){
30             result.add(new ArrayList<>(current)); // copy!
31         }
32
33         dfs(root.left, target - root.val, current, result);
34         dfs(root.right, target - root.val, current, result);
35
36         current.remove(current.size() - 1); // backtrack
37     }
38 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3

Input

root = [5,4,8,11,null,13,4,7,2,null,null,5,1]

targetSum =